FORD-FULKERSON ALGORITHM

Tamim Hasan Saad
Student ID: 2005095
Habiba Rafique
Student ID: 2005096
Sadia Afrin Sithi
Student ID: 2005119

CSE-300:Technical Writing and Presentation
Department of Computer Science and Engineering
Bangladesh University of Engineering and Technology

March 9, 2024

Contents

1 INTRODUCTION **1**

2 FORD FULKERSON METHOD **1**
2.1 **DEFINTIONS AND CONCEPTS** 2
2.2 **PROBLEM SPECIFICATION** 5

3 FORD-FULKERSON ALGORITHM **6**
3.1 **ALGORITHM DESCRIPTION** 6
3.2 **FLOW-CHART** 8

4 EXAMPLE **9**

5 MAXFLOW-MINCUT **10**
5.1 MIN-CUT 10
5.2 **MAXFLOW-MINCUT THEOREM** 10

6 ANALYSIS **11**

7 RUNNING TIME **11**

8 CONCLUSION **12**
8.1 SUMMARY 12
8.2 STRENGTHS AND WEAKNESS 12

9 ACKNOWLEDGEMENT **13**

List of Figures

1 Residual Graph 3
2 Augmenting Path and Bottleneck Capacity 4
3 Flowchart depicting the Ford-Fulkerson algorithm. 8
4 Maxflow-Mincut Algorithm. 10

List of Tables

1 Relation between path, capacity, and Resulting Flow Network 9

1 INTRODUCTION

The Ford-Fulkerson algorithm is a classic method for computing the maximum flow in a flow network, which is a directed graph where each edge has a capacity and represents a pathway through which a material (or flow) can travel from a source node to a sink node. This algorithm iteratively finds augmenting paths from the source to the sink and increases the flow along these paths until no more augmenting paths can be found.

2 FORD FULKERSON METHOD

The Ford–Fulkerson method or Ford–Fulkerson algorithm (FFA) is a **greedy algorithm** that computes the maximum flow in a flow network. It is called a "method" instead of an "algorithm" as the approach to finding augmenting paths in a residual graph is not fully specified or it is specified in sev-eral implementations with different running times. It was pub-lished in 1956 by **L.R. Ford Jr.** and **Dr. Fulkerson**. The name "Ford–Fulkerson" is often also used for the Edmonds-Karp algorithm, which is a specialization of Ford–Fulkerson. The idea behind the algorithm is as follows: as long as there is a path from the source (start node) to the sink (end node), with available capacity on all edges in the path, we send flow along one of the paths. Then we find another path, and so on. A path with available capacity is called an augmenting path.

The Ford-Fulkerson algorithm particularly has a lot of applica-tions. Some of them are:

- Network Traffic Optimization
- Water Distribution Systems
- Electric Power Grids
- Airline scheduling
- stereo cor-respondence etc.

where the given problem is transformed into a maximum flow minimum cut problem and then solved us-ing the Ford-Fulkerson algorithm. [1]

2.1 DEFINTIONS AND CONCEPTS

1. **Flow Network**
   - A flow network is a directed graph G = (V,E) where each edge (u,v) has a non-negative capacity c(u,v), representing the maximum flow that can pass through that edge.

2. **Flow**
   - A flow in a network is a function $\boldsymbol{f : V \times V \to \mathbb{R}}$ that satisfies the properties: $\boldsymbol{0 \leq f(u,v) \leq ¸u,v)}$ for all (u, v) in E known as **Capacity constraint** and For every node $\boldsymbol{v \in V - \{s,t\}}$, the total incoming flow is equal to the total outgoing flow: $\boldsymbol{\sum_{u \in V} f(u,v) = \sum_{w \in V} f(v,w)}$ known as **Flow conservation**.

3. **Maximum Flow**
   - Maximum flow is the maximum amount of flow that can be sent from the source node s to the sink node t in the flow network.
   - It is achieved when no more augmenting paths can be found in the residual graph.
   - The flow obtained using the Ford-Fulkerson algorithm is considered the maximum flow for the given network.

4. **Residual Graph**
   - A residual graph is a modified version of the original flow network graph.
   - It reflects the remaining capacity on each edge after some flow has been pushed through.
   - In the residual graph:
     - If the original edge has capacity $c(u, v)$ and flow $f(u, v)$, the residual capacity of the edge is $\boldsymbol{c(u,v) - f(u,v)}$.
     - If there's a reverse edge in the original graph (to allow backflow), with flow $f(v, u)$, the residual capacity of this edge is $\boldsymbol{f(v,u)}$.
   - Residual graphs are used to find augmenting paths efficiently in the Ford-Fulkerson algorithm.

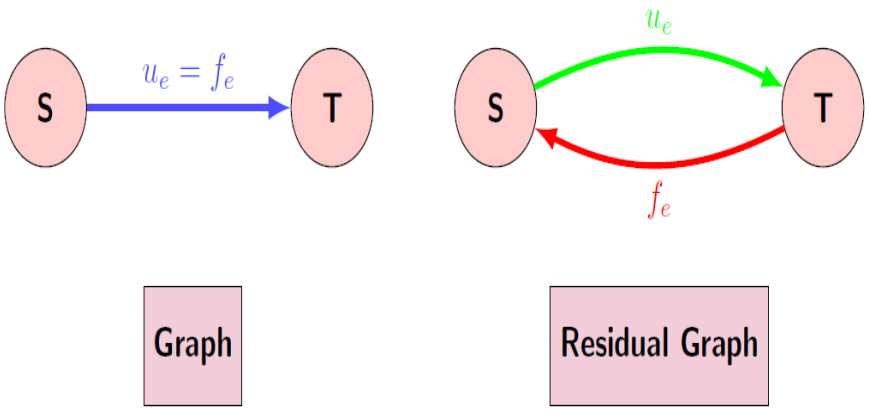


Figure 1: Residual Graph

5. **Augmenting Path**

- An augmenting path is a simple path from the source node s to the sink node t in the residual graph.
- Residual graph is a modified version of the original graph that reflects the remaining capacity on each edge after the flow has been pushed through some paths.
- An augmenting path allows us to increase the flow from the source to the sink.
- When no augmenting path exists, the flow obtained is considered maximum.

6. **Bottleneck Capacity**

- The bottleneck capacity of an augmenting path is the minimum capacity among all the edges in that path.
- It determines the maximum amount of flow that can be pushed through the augmenting path.
- When augmenting the flow along a path, the bottleneck capacity limits the amount of flow that can be increased.
- The bottleneck capacity is crucial in determining the maximum flow through the network.

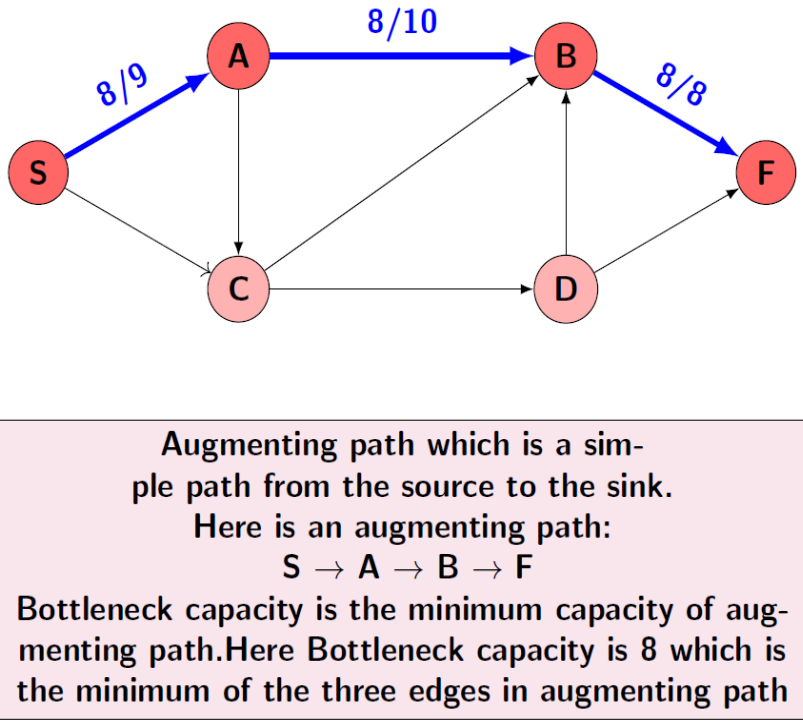


Figure 2: Augmenting Path and Bottleneck Capacity

These definitions provide a basic understanding of the key elements involved in the Ford-Fulkerson algorithm for computing maximum flow in a flow network.

2.2 PROBLEM SPECIFICATION

Given a flow network represented by a directed graph G = (V,E) with a source node s, a sink node t, and capacities c(u,v) on each edge $(u, v) \in E$, the Ford-Fulkerson algorithm aims to find the maximum possible flow from the source s to the sink t while respecting the capacity constraints of the edges.

More formally, the problem can be stated as follows:

1. **Input**
   - A directed graph G = (V,E) representing the flow network.
   - Source node s and sink node t in V.
   - Capacities c(u,v) for each edge $(u, v) \in E$, indicating the maximum flow that can traverse that edge.
2. **Output**
   - A flow function f that assigns a flow value to each edge $(u, v) \in E$, such that:
     (a) **Capacity constraint:** $0 \leq f(u, v) \leq c(u, v)$ for all $(u, v) \in E$.
     (b) **Flow conservation:** For every node $v \in V - \{s, t\}$, the total incoming flow is equal to the total outgoing flow:

$$\sum_{u \in V} f(u, v) = \sum_{w \in V} f(v, w)$$

   - The maximum flow value, which represents the total amount of material that can be transported from s to t in the network.

The Ford-Fulkerson algorithm iteratively finds augmenting paths from s to t and increases the flow along these paths until no more augmenting paths exist. An augmenting path is a simple path from s to t in the residual graph (a modified version of the original graph that reflects the remaining capacity on each edge). When no more augmenting paths can be found, the algorithm terminates, and the flow obtained is the maximum flow. [2]

3 FORD-FULKERSON ALGORITHM

3.1 ALGORITHM DESCRIPTION

Let $G(V, E)$ be a graph, and for every edge from u to v let $c(u, v)$ be the capacity and $f(u, v)$ be the flow. We want to find the maximum flow from the source s to sink t. After every step in the algorithm, the following conditions are maintained:

- The flow along an edge cannot exceed its capacity.
- The net flow from u to v must be the opposite of the net flow from v to u.
- That is, unless u is s or t, the net flow to a node is zero except for the source which produces the flow and the sink, which consumes flow.
- The flow leaving from s must be equal to the one arriving at t.
- **Capacity Constraints:** $\forall(u, v) \in f(u, v) \leq c(u, v)$
- **Skew Symmetry:** $\forall(u, v) \in f(u, v) = -f(u, v)$
- **Flow Conservation:** $\forall u \in V : u \neq s$ and $u \neq t, \sum_{w \in V} f(u, w) = 0$, (where $w \in v$)
- **Value(f):** $\sum f(s, u) = \sum f(v, t)$ where $s, u, v, t \in E$

This means that the flow through the network is a legal flow after each round in the algorithm. We define the residual network $G_f(V, E_f)$ to be the network

with capacity $c_f(u, v) = c(u, v) - f(u, v)$. Notice that it can happen that a flow from v to u is allowed in the residual network, though not allowed in the original network:

$$\begin{aligned} \text{If } f(u, v) > 0 \text{ and } c(u, v) = 0, & \\ \text{then } c_f(v, u) &= c(v, u) - f(v, u) \\ &= f(u, v) > 0. \end{aligned}$$

The basic algorithm is as follows:

Repeat: find an augmenting path, and augment!

Algorithm 1 Ford-Fulkerson Algorithm

```
1: For each edge (u, v) ∈ G.E, f(u, v) = 0.
2: while there exists a path p from s to t in the residual network do
3:     c_f(p) = min{c_f(u, v) : (u, v) is in p}
4:     for each edge (u, v) ∈ p do
5:         f(u, v) = f(u, v) + c_f(p)        ▷ Send flow along the path
6:         f(v, u) = f(v, u) − c_f(p)        ▷ The flow might be returned later
7:     end for
8: end while
```

The path in the while loop can be found with a breadth-first or depth-first search in $G_f(V, E_f)$. When the former is used, the algorithm is called Edmonds-Karp algorithm.

When no more paths in the while loop can be found, s will not be able to reach t in the residual network. If S is the set of nodes reachable by s in the residual network, then the total capacity in the original network of edges from S to the remainder of V is, on the one hand, equal to the total flow we found from s to t, and on the other hand, serves as an upper bound for all such flows. This proves that the flow found is maximal.

3.2 FLOW-CHART

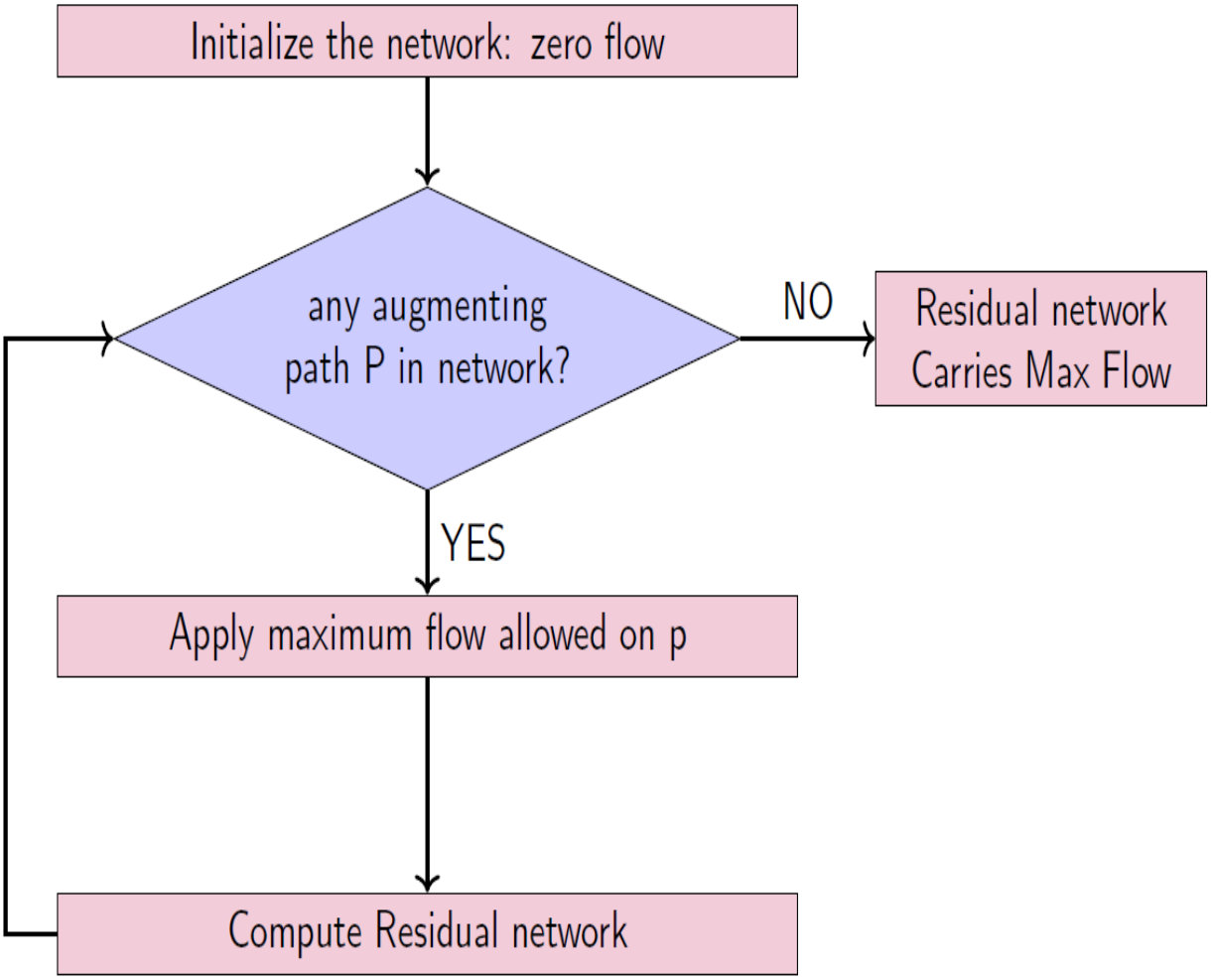


Figure 3: Flowchart depicting the Ford-Fulkerson algorithm.

4 EXAMPLE

The following example shows the first steps of Ford Fulkerson in a flow network with 6 nodes, source S, sink F.

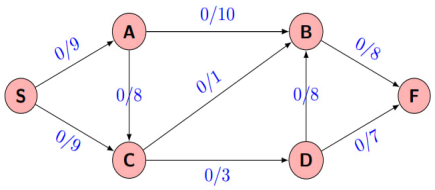
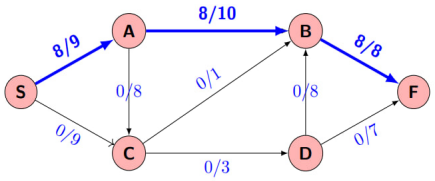
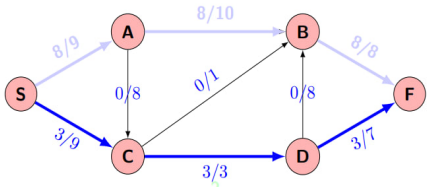
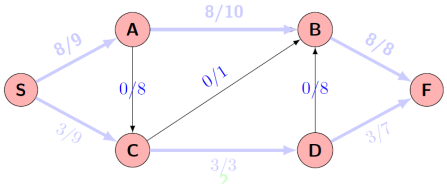
PATH	CAPACITY	RESULTING NETWORK
Initial Network		
S,A,B,F	$min(c_f(S,A), c_f(A,B), c_f(B,F))$ $= min(c(S,A) - f(S,A), c(A,B) - f(A,B), c(B,F) - f(B,F))$ $= min(9-0, 10-0, 8-0) = 8$	
S,C,D,F	$min(c_f(S,C), c_f(C,D), c_f(D,F))$ $= min(c(S,C) - f(S,C), c(C,D) - f(C,D), c(D,F) - f(D,F))$ $= min(9-0, 3-0, 7-0) = 3$	
Resulting network		

Table 1: Relation between path, capacity, and Resulting Flow Network

5 MAXFLOW-MINCUT

5.1 MIN-CUT

Min-cut is the one with the minimum total capacity. It has the smallest sum of capacities among all cuts that separate the source and sink nodes.
The value of the minimum cut is equal to the maximum flow in the network, which is a fundamental property exploited by the Ford-Fulkerson algorithm for finding the maximum flow.

5.2 MAXFLOW-MINCUT THEOREM

If f is a flow in a flow network G = (V,E), with source s and sink t, then the following conditions are equivalent:

$\Rightarrow$ f is a maximum flow in G.

$\Rightarrow$ The residual network G_f contains no augmented paths.

$\Rightarrow$ $|f| = c(S,T)$ for some cut (S,T) (a min-cut).

It is a flow since there is no augmented paths It is maximum since the sink is not reachable from the source.

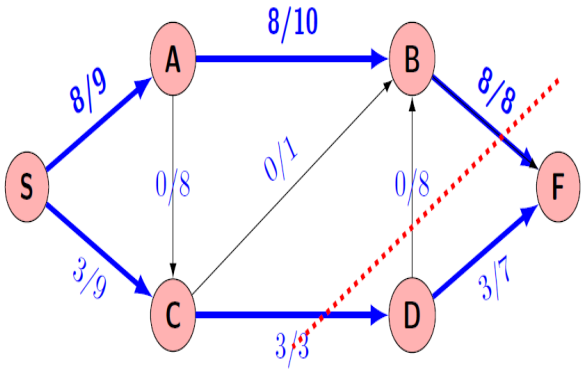


Figure 4: Maxflow-Mincut Algorithm.

6 ANALYSIS

The efficiency of the **Ford-Fulkerson algorithm** heavily relies on how the augmenting paths **p** are chosen. Poor choices can lead to non-termination or failure to converge to the maximum flow value.

This issue is particularly pronounced when dealing with irrational capacities. In practice, the maximum-flow problem often involves integral capacities, and if they're rational, they can be scaled to integers without loss of generality. In the transformed network with integral capacities, denoted by **f***, where **f*** represents the **maximum flow**, a basic implementation of Ford-Fulkerson executes the while loop (lines 2-8) at most $|f^*|$ times. This is because the flow value increases by at least one unit in each iteration.

For graphs with integral capacities, the runtime of Ford-Fulkerson is bounded by **O(Ef)**, where E is the number of edges and f is the maximum flow. However, in the presence of irrational values, the flow may fail to converge towards the maximum value, leading the algorithm to run indefinitely.

7 RUNNING TIME

Complexities:

- Residual path: $O(m + n)$
- Compute bottleneck capacity: $O(m)$
- Update flow: $O(m)$
- Update residual graph: $O(m)$

Total running time:

$$O(C(m + n))$$

8 CONCLUSION

8.1 SUMMARY

We have discussed the **Ford-Fulkersion** method for maximum flow and minimum cut and analysed it's algorithm.Ford-Fulkerson algorithm is a versatile tool with numerous real-world applications in optimizing flow management across various domains.In essence, the Ford-Fulkerson algorithm is a pivotal tool in network optimization, renowned for its versatility and effectiveness in efficiently managing flow and resource allocation across various real-world applications.

8.2 STRENGTHS AND WEAKNESS

- The Ford-Fulkerson algorithm possesses several strengths that make it a valuable tool in various optimization problems. Firstly, its versatility allows it to be applied to a wide range of flow network scenarios, including transportation systems, network routing, and resource allocation. Additionally, the algorithm's ability to handle both integral and rational capacities makes it adaptable to real-world scenarios where flow capacities may vary. Moreover, its iterative approach ensures that it converges to the maximum flow solution efficiently, particularly in networks with integral capacities.
- However, despite its strengths, the Ford-Fulkerson algorithm has some limitations. One significant weakness is its reliance on finding augmenting paths, which may lead to non-termination or slow convergence, especially in networks with irrational capacities. Additionally, the algorithm's performance can be impacted by the choice of path selection strategy, as poor choices may result in suboptimal solutions or prolonged runtime. Overall, while the Ford-Fulkerson algorithm offers a powerful framework for solving maximum flow problems, its effectiveness can be influenced by the specific characteristics of the flow network and the implementation choices made during execution.

9 ACKNOWLEDGEMENT

We would like to express our sincere gratitude to the Department of Computer Science and Engineering of Bangladesh University Of Engineering and Technology for their unwavering support and encouragement throughout the duration of this semester. Their guidance and resources have been instrumental in our endeavors, enabling us to delve into the complexities with confidence and diligence.

References

[1] Thomas H Cormen, Charles E Leiserson, Ronald L Rivest, and Clifford Stein. Section 26.2: The ford-fulkerson method. *Introduction to algorithms*, pages 651–664, 2001.

[2] Jon Kleinberg and Eva Tardos. *Algorithm design*. Pearson Education India, 2006.